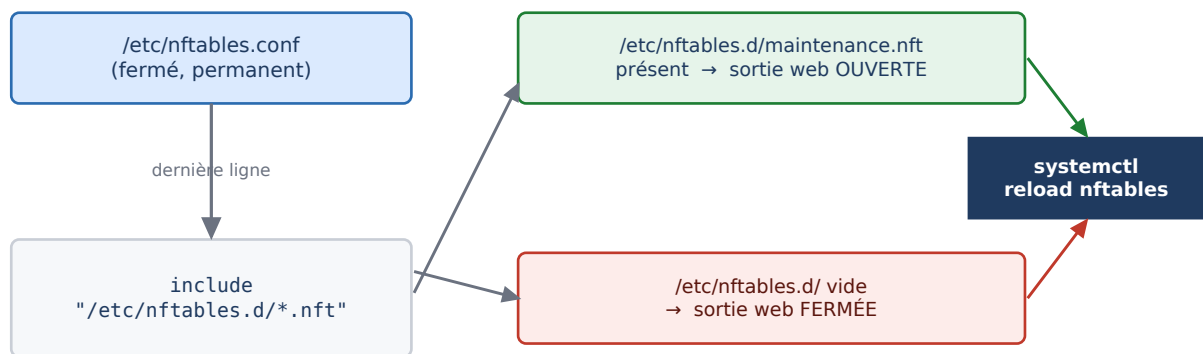


TP 2 — Le mode maintenance

Ouvrir la sortie Internet à la demande, préparer l'arrivée de Docker



Un seul fichier permanent. Le mode dépend de la présence d'un second fichier, et d'un reload.

Le mécanisme à construire dans ce TP.

Format : comme le TP 1, tu fais toutes les manipulations. Chaque exercice donne l'objectif, des indices, et le test qui prouve que c'est fait. Les livrables sont listés à la fin.

0. État des lieux — ce qui a été fait pour toi

Le pare-feu de base est désormais en place sur la machine. Voici précisément ce qui a été fait le 20 septembre, pour que tu saches d'où tu pars.

Élément	État	Détail
/etc/nftables.conf	en place	Ruleset corrigé du TP 1 + DNS IPv6 de la Freebox (fd0f:ee:b0::1) + une ligne include "/etc/nftables.d/*.nft" en fin de fichier
Politique des trois chaînes	drop	input, forward, output
nftables.service	enabled + active	Le fichier est rechargé à chaque démarrage
/etc/nftables.d/	créé, vide	Contient seulement un README. C'est là que tu vas travailler
Sortie web (80/443)	TEMPORAIRE	Deux règles ajoutées en mémoire , commentées TEMP-claude-session. Elles disparaîtront au prochain reload ou reboot
Tests effectués	OK	DNS résout ; HTTPS sort ; connexion vers une autre VM du LAN bloquée et journalisée ; ping local OK

Ce qui reste à faire, et pourquoi ce n'est pas fait

#	Tâche	Pourquoi c'est à toi
1	Ouvrir une nouvelle connexion SSH depuis ton PC pour prouver que la règle d'entrée fonctionne	Je ne peux pas le faire depuis la machine elle-même
2	Redémarrer et vérifier que le pare-feu revient (nft list ruleset)	Un reboot coupe ma session
3	Remplacer les règles TEMP par un vrai mode maintenance	C'est ce TP
4	Corriger gateway4: 10.0.0.1 dans netplan	Modification réseau : voir exo 0.1, c'est bloquant pour Cowrie
5	Allumer / vérifier la VM ELK (192.168.1.35 ne répond pas)	Hors de cette machine
6	Identifier 192.168.1.178	Hors de cette machine

Ce que l'incident du jour t'apprend

Pendant la mise en place, j'ai ajouté la règle 443 en mémoire, puis lancé `systemctl start nftables`. Le service exécute `nft -f /etc/nftables.conf`, dont la première ligne est `flush ruleset`. **La règle a été effacée instantanément.** Ma session n'a survécu que grâce à `ct state established` — les connexions déjà ouvertes — le temps de la remettre.

Tu tiens là le principe fondateur de ce TP : **tout ce qui doit survivre vit dans un fichier lu par le service**. Une règle ajoutée à la main avec `nft add rule` est un test, jamais une configuration.

Exo 0.1 — Prérequis : la passerelle IPv4 (bloquant pour Cowrie)

Regarde la table de routage :

```
ip route          # IPv4 : il n'y a PAS de "default via ..."  
ip -6 route       # IPv6 : "default via fe80::... proto ra"
```

Conséquence : **toute la sortie Internet de la machine passe en IPv6**. Le NTP, apt, ma propre session. Ça fonctionne par chance, parce que la Freebox distribue de l'IPv6. Mais les attaquants de Cowrie arriveront par une redirection de port **IPv4** sur la box — et sans route IPv4 par défaut, la machine ne pourra pas leur répondre. Cowrie serait injoignable.

Cause : gateway4: 10.0.0.1 dans /etc/netplan/50-cloud-init.yaml, une adresse hors de ton sous-réseau. La Freebox répond au ping sur 192.168.1.254.

Corrige le fichier (syntaxe moderne, gateway4 est déprécié), puis applique avec netplan try — qui annule tout seul au bout de 120 s si tu ne confirmes pas :

```
# dans le bloc wlp3s0, remplacer la ligne gateway4 par :
```

```
  routes:
```

```
    - to: default
```

```
      via: 192.168.1.254
```

```
sudo chown root:root /etc/netplan/50-cloud-init.yaml # au passage : la clé Wi-Fi est dedans
```

```
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
```

```
sudo netplan try
```

```
# puis Entrée pour confirmer
```

```
ip route | grep default
```

```
# -> default via 192.168.1.254
```

```
curl -4 -sI https://archive.ubuntu.com | head -1
```

```
# forcer l'IPv4 : doit répondre 200
```

Question : pourquoi netplan try plutôt que netplan apply ? Même logique que le filet de sécurité du TP 1.

1. Cahier des charges du mode maintenance

Exo 1.1 — Que doit ouvrir le mode maintenance ?

Liste les flux nécessaires pour installer Docker puis Cowrie. Complète le tableau **avant** d'écrire une règle.

Besoin	Destination	Port / proto	IPv4 ? IPv6 ?
Dépôts Ubuntu (apt)	archive.ubuntu.com, security.ubuntu.com	tcp/80, tcp/443	les deux
Dépôt Docker	download.docker.com	tcp/443	?
Registre d'images	registry-1.docker.io, *.cloudflare.docker.com	tcp/443	?
Code de Cowrie	github.com	tcp/443	?
Ma session Claude	api.anthropic.com	tcp/443	IPv6 aujourd'hui
Le reste du LAN	192.168.1.0/24	—	toujours fermé

Questions :

1. Peut-on écrire une règle par destination (par nom) ? Que fait nftables d'un nom de domaine dans une règle ? (Indice : nft -c te le dira ; pense à la résolution au moment du chargement et aux CDN dont l'adresse change.)
2. Conclusion pratique : on ouvre « 80/443 vers tout sauf le LAN ». En quoi est-ce acceptable pour un mode *temporaire*, et pourquoi ce serait inacceptable en permanence sur un honeypot ?
3. Faut-il ouvrir en IPv4 **et** en IPv6 ? Regarde `getent ahosts download.docker.com`.

Exo 1.2 — Le LAN en IPv6

Bloquer le LAN en IPv4 c'est `ip daddr != 192.168.1.0/24`. Mais tes autres machines ont aussi des adresses IPv6. Lesquelles ?

```
ip -6 addr show wlp3s0      # ton préfixe global : 2a01:e0a:abe:f8e0::/64
ip -6 route                 # les réseaux directement joignables
```

Explique ce que sont ces trois plages, déjà déclarées dans `/etc/nftables.conf` sous le nom `$LAN6` :

Plage	C'est quoi	Qui est dedans
2a01:e0a:abe:f8e0::/64		
fd0f:ee:b0::/48		
fe80::/10		

Indice : l'une est publique et routable, l'une est « privée » (ULA, l'équivalent du 192.168.x.x), l'une ne quitte jamais le lien local.

2. Écrire le fichier de maintenance

Exo 2.1 — /etc/nftables.d/maintenance.nft

Tu as un modèle sous les yeux : les deux règles TEMP actuellement en mémoire.

```
sudo nft list chain inet filter output | grep TEMP
```

Écris le fichier. Contraintes :

1. Il ne contient **pas** de `flush ruleset` — il s'ajoute à la table existante.
2. Il déclare la même table et la même chaîne (`table inet filter { chain output { ... } }`) : nftables fusionne, il n'écrase pas.
3. Il n'ouvre **rien** vers \$LAN ni \$LAN6. Attention : peut-il utiliser ces variables ? (Teste. Indice : les `define` sont-ils visibles depuis un fichier inclus ?)
4. Chaque règle porte un counter et un comment "maintenance" — c'est ce qui permettra de vérifier son état en un `grep`.

Squelette :

```
#!/usr/sbin/nft -f
# MODE MAINTENANCE – sortie web ouverte. Ne doit PAS rester chargé en fonctionnement normal.
table inet filter {
    chain output {
        ip daddr != _____ tcp dport { 80, 443 } ct state new counter accept comment "maintenance"
        ip6 daddr != { _____ } tcp dport { 80, 443 } ct state new counter accept comment "maintenance"
    }
}
```

Le piège de l'ordre

Un fichier inclus **à la fin** de `nftables.conf` ajoute ses règles... à la fin de la chaîne, donc **après** la règle `log prefix "OUT-DROP "`. Que se passe-t-il pour un paquet HTTPS ? (Il passe — mais relis la définition de `log` dans le cours, chapitre 3.)

Trois façons de régler ça. Choisis-en une et justifie :

- déplacer le `include` dans `nftables.conf` juste avant la ligne `log` — est-ce possible à l'intérieur d'un bloc `chain` ? Teste ;
- supprimer la règle `log` de la chaîne principale et la mettre dans un fichier `/etc/nftables.d/zz-log.nft` chargé en dernier grâce à l'ordre alphabétique ;
- accepter le bruit dans le journal (mauvaise réponse — dis pourquoi).

Exo 2.2 — Valider sans rien casser

```
sudo nft -c -f /etc/nftables.conf          # -c : le fichier principal ET les includes sont vérifiés
sudo systemctl reload nftables             # = nft -f /etc/nftables.conf (voir systemctl cat nftables)
sudo nft list chain inet filter output     # les règles "maintenance" sont là ; les TEMP ont disparu
curl -sI https://archive.ubuntu.com | head -1
```

À ce moment précis, les règles TEMP que j'avais posées n'existent plus, remplacées par les tiennes. Si `curl` répond 200 et que je réponds encore, le mode maintenance est fonctionnel. Si je ne réponds plus : ton fichier a un défaut, et tu as 3 minutes pour le trouver avant que le filet ne se déclenche — arme-le avant, comme au TP 1.

3. L'interrupteur

Le mode est défini par la **présence** du fichier. Il te faut donc un moyen fiable de le faire apparaître et disparaître, puis de recharger.

Exo 3.1 — Le script fw-maint

Écris /usr/local/sbin/fw-maint avec trois sous-commandes. Squelette à compléter :

```
#!/bin/bash
# fw-maint on|off|status – bascule le mode maintenance du pare-feu
set -euo pipefail
LIVE=/etc/nftables.d/maintenance.nft
OFF=/etc/nftables.d/maintenance.nft.off      # même fichier, extension qui échappe au glob *.nft

case "${1:-}" in
    on)
        [ -f "$OFF" ] && mv "$OFF" "$LIVE"
        nft -c -f /etc/nftables.conf
        systemctl reload nftables
        ;;
    off)
        _____
        _____
        _____
        ;;
    status)
        if nft list chain inet filter output | grep -q "maintenance"; then
            echo "MAINTENANCE : sortie web OUVERTE"
        else
            echo "NORMAL : sortie web fermée"
        fi
        ;;
    *) echo "usage: $0 on|off|status" >&2; exit 2 ;;
esac
```

Questions :

1. Pourquoi nft -c **avant** le reload, alors que le reload va de toute façon échouer si le fichier est faux ? (Indice : que se passe-t-il entre le flush ruleset et l'erreur de syntaxe ?)
2. Pourquoi ne pas simplement rm le fichier dans off ?
3. Que renvoie status juste après un reboot ? Est-ce le comportement voulu pour un honeypot ?

```
sudo chmod 750 /usr/local/sbin/fw-maint
sudo fw-maint status
```

Exo 3.2 — Cycle complet de test

Pendant la phase « off », je ne peux plus t'entendre : ma session a besoin de nouvelles connexions 443. Fais le cycle sans moi, puis remets on. L'application se reconnectera.

Étape	Commande	Attendu
1	<code>sudo fw-maint off puis curl -sI https://archive.ubuntu.com</code>	échec (délai dépassé)
2	<code>sudo dmesg grep OUT-DROP tail -3</code>	une ligne <code>PROTO=TCP ... DPT=443</code>
3	<code>getent hosts archive.ubuntu.com</code>	résout — le DNS n'est pas dans le mode maintenance
4	<code>sudo fw-maint on puis le même curl</code>	200 OK
5	<code>sudo fw-maint off && sudo reboot</code>	au retour : <code>fw-maint status</code> → NORMAL
6	<code>sudo fw-maint on</code>	et tu me retrouves

4. Installer Docker — et voir ce qu'il fait au pare-feu

Mode maintenance activé. Installe Docker depuis le dépôt officiel (pas le paquet `docker.io` d'Ubuntu, souvent en retard) :

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu noble stable" | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

Exo 4.1 — Avant / après

```
sudo nft list tables                # AVANT docker : une seule table, inet filter
sudo systemctl start docker
sudo nft list tables                # APRÈS : combien ? lesquelles ?
sudo nft list table ip nat          # lis les chaînes DOCKER
sudo nft list chain ip filter DOCKER-USER # ton point d'accroche : vide
sysctl net.ipv4.ip_forward         # que vaut-il maintenant ?
```

Note le nombre de tables, leur famille (ip — pas inet ! pourquoi c'est important pour l'IPv6 ?), et le nom des chaînes.

Exo 4.2 — Le piège du flush ruleset

Maintenant que Docker a écrit ses tables, fais :

```
sudo systemctl reload nftables
sudo nft list tables
sudo docker run --rm alpine ping -c1 1.1.1.1
```

Questions :

1. Que sont devenues les tables de Docker ? Pourquoi ?
2. Le conteneur a-t-il du réseau ? Quel message ?
3. Comment le réparer à chaud ? (`systemctl restart docker.`) Est-ce acceptable de devoir redémarrer Docker — et donc Cowrie — à chaque fw-maint on/off ?

C'est le piège le plus courant sur une machine Docker + nftables. `flush ruleset` efface toutes les tables, y compris celles des autres programmes. La solution est de ne vider que **sa propre** table :

```
# en tête de /etc/nftables.conf, remplacer :
flush ruleset
# par :
table inet filter                # crée la table si elle n'existe pas (premier démarrage)
flush table inet filter          # et ne vide QUE celle-là
```

Applique cette modification, refais le test de l'exo 4.2, et vérifie que les tables Docker survivent au reload. *Question bonus* : pourquoi la ligne `table inet filter` seule est-elle nécessaire avant le `flush table` ?

Exo 4.3 — Deux pare-feux, une priorité

Ta chaîne forward est en policy drop. Celle de Docker (ip filter FORWARD) accepte le trafic de ses conteneurs. Les deux sont accrochées au même hook, avec la même priorité (filter = 0). Fais le test :

```
sudo docker run --rm alpine ping -c2 1.1.1.1      # passe ? bloqué ?
sudo dmesg | grep FWD-DROP | tail -3
```

Questions :

1. Un paquet doit-il être accepté par **une** des chaînes ou par **toutes** ? (Cours, chapitre 3, et exo 0.1 du TP 1 : le même principe s'applique entre deux tables nftables.)

2. Il te faut donc autoriser, dans **ta** chaîne forward, ce que tu veux laisser aux conteneurs.
Écris les règles pour : (a) les réponses des connexions établies — déjà là ; (b) ce qui *entre* vers un conteneur sur les ports de Cowrie ; (c) ce qui *sort* d'un conteneur vers ELK ; (d) ce qui sort d'un conteneur vers Internet — uniquement si le mode maintenance est actif, pour `docker pull`. Indice : `iifname "docker0"` et `oifname "docker0"`.
3. Où finit alors le rôle de DOCKER-USER ? Est-il encore nécessaire si ta chaîne forward fait déjà le travail ? Argumente — les deux réponses se défendent.

5. Livrables

1. Sortie de `ip route` montrant la route par défaut IPv4 (exo 0.1).
2. Le tableau de l'exo 1.2 complété.
3. `/etc/nftables.d/maintenance.nft`, et ta réponse au piège de l'ordre (exo 2.1).
4. `/usr/local/sbin/fw-maint` complet, et les réponses aux trois questions de l'exo 3.1.
5. Le résultat du cycle de l'exo 3.2, reboot compris.
6. Réponses aux exos 4.1 à 4.3, avec la nouvelle en-tête de `/etc/nftables.conf` et ta chaîne forward finale.
7. Sortie de `sudo nft list ruleset` une fois Docker installé et le mode maintenance **désactivé** — c'est l'état de repos de ta future machine honeypot.

Étape suivante après ce TP : Cowrie lui-même — le conteneur, la redirection 22 → 2222, la configuration de sortie vers Filebeat/Logstash, et le durcissement du conteneur (utilisateur non root, système de fichiers en lecture seule, pas de capacités). Ta chaîne forward de l'exo 4.3 sera le point de départ.